Flutter Theming App - Detailed Project Report

This report explains the development of a Flutter application focused on theming, Material 3 design, and dynamic light/dark mode support. The application demonstrates custom color schemes, reusable themes, responsive UI components, and theme persistence using SharedPreferences.

1. Learning Objectives

- Understand Flutter theming
- Implement custom themes
- Create light and dark modes
- Style the entire application
- Learn Material 3 theming
- Use Theme.of(context) for consistent styling

2. Technologies Used

- Flutter SDK
- Dart Programming Language
- Material 3 Design System
- SharedPreferences Package
- ThemeData and ColorScheme

3. Learning Resources

YouTube Search Terms

- Flutter theming complete guide
- Flutter dark mode tutorial
- Flutter custom theme
- Material 3 theming Flutter
- Flutter color schemes

Recommended Channels

- Flutter Official
- Reso Coder
- The Flutter Way

Official Documentation: https://docs.flutter.dev/cookbook/design/themes

4. Application Overview

The application named 'Theme Studio' demonstrates advanced Flutter theming concepts. It allows users to switch between System, Light, and Dark themes using a segmented control. The app uses Material 3 components and dynamically updates colors and typography across the UI.

5. Main Features

- Custom Material 3 themes
- Light and Dark mode support
- ThemeMode.system integration
- Persistent theme selection using SharedPreferences
- Dynamic color palette
- Reusable themed widgets
- Modern gradient-based UI
- Segmented theme switcher
- Consistent typography and spacing

6. Understanding ThemeData

- ThemeData controls the visual appearance of the Flutter application.
- It manages colors, typography, buttons, cards, app bars, and component styling.
- Using ThemeData ensures consistent UI design throughout the app.

7. Custom Color Scheme

- The application uses ColorScheme.fromSeed() for generating Material 3 color palettes.
- Primary brand color is teal (0xFF00695C).
- Separate light and dark color schemes are defined.
- Semantic colors are used for better readability and accessibility.

8. Typography and Text Styling

- Typography.material2021() is used for Material 3 text styles.
- Light theme uses black typography.
- Dark theme uses white typography.
- Text styles are accessed using Theme.of(context).textTheme.

9. Theme Switching Functionality

- Users can switch between System, Light, and Dark modes.
- SegmentedButton widget is used for selecting themes.
- Theme preferences are stored using SharedPreferences.
- Theme selection persists even after restarting the app.

10. Material 3 Theming

- useMaterial3: true is enabled in ThemeData.
- Modern Material 3 surfaces and elevations are used.
- Dynamic surface tint colors improve visual depth.
- Material 3 typography and color roles are applied.

11. Widgets Used in the Application

- MaterialApp

- Scaffold
- CustomScrollView
- SliverAppBar
- SegmentedButton
- Container
- Wrap
- Card
- Text
- Column and Row
- Theme.of(context)

12. Custom Widgets

- _HeroCard widget for showcasing themed content
- _ColorTile widget for displaying theme colors
- _FeatureCard widget for application features

13. Application Flow

1. App initializes SharedPreferences.
2. Saved theme preference is loaded.
3. MaterialApp applies light or dark theme.
4. User changes theme using SegmentedButton.
5. Theme selection updates UI instantly.
6. Preference is saved locally.

14. Tasks Completed

- Watched Flutter theming tutorials
- Read official Flutter documentation
- Implemented ThemeData
- Created custom themes
- Implemented light and dark themes
- Added ThemeMode.system
- Built reusable widgets
- Created a custom color palette
- Styled all application components

15. Deliverables

- Themed Flutter application
- Custom color scheme
- Light and dark modes
- Theme switcher
- All components themed
- README with theming guide

16. Conclusion

This project provided practical experience in Flutter theming and Material 3 design implementation. The application successfully demonstrates dynamic theming, reusable styling, persistent preferences, and consistent UI design practices. It improved understanding of ThemeData, ColorScheme, typography, and Flutter widget styling.